

P2652R1: Disallow User Specialization of `allocator_traits`

Pablo Halpern <phalpern@halpernwrightsoftware.com>

2022-11-11 09:32 HST

Target audience: LWG

Abstract

The `allocator_traits` class template was introduced in C++11 with two goals in mind: 1) Provide default implementations for allocator types and operations, thus minimizing the requirements on allocators [`allocator.requirements`], and 2) provide a mechanism by which future standards could extend the allocator interface without changing allocator requirements and thus obsoleting existing allocators. The latter goal is undermined, however, by the standard currently allowing user-defined specializations of `std::allocator_traits`. Although the standard requires that any such specialization conform to the standard interface, it is not practical to change the standard interface – even by extending it – without breaking any existing user specializations. Indeed, the Sep 2022 C++23 CD, [N4919](#) contains an extension, `allocate_at_least`, that logically belongs in `std::allocator_traits`, but is expressed as an unrelated function because of the problem of potential user-defined specializations.

This paper proposes that the standard remove the user's latitude for specializing `std::allocator_traits`.

This paper is the proposed resolution to a US NB comment having the same title; it is targeted for C++23.

Change History R0 -> R1

R0 was reviewed by LEWG at the Kona meeting on 2022-11-09 and the first proposed resolution (of the two PRs in R0) was forwarded to LWG with no changes. LWG asked for the changes in this version on 2022-11-10 during the same Kona meeting.

- There was no interest in LEWG in proposed resolution 2, so it was removed.
- Changed the count in `allocation_result` from `std::size_t` to the allocator's `size_type`. Although not directly related to this proposal, LWG noticed this oversight while reviewing the paper and asked for this change.
- Changed size argument for `allocate_at_least` from `size_t` to `size_type`.

Motivation

The minimal interface for a type conforming to the allocator requirements is that it have a `value_type` type, `allocate` and `deallocate` member functions, and equality comparison operators. The `allocator_traits` class template provides many other types and functions such as `pointer`, `rebind`, and `construct`. Generic types that use allocators are required to access the allocator through `std::allocator_traits`. The latter requirement was intended to allow the allocator interface to be extended without necessarily changing every existing allocator.

For example, C++03 allocators did not have a `void_pointer` member, but such a member is provided automatically through `allocator_traits`; an allocator class can override the default provided by `allocator_traits`, but is not required to do so.

The Standard description for each trait `x` in `std::allocator_traits<A>` typically follows the form, “`a.x` if that expression is well-formed; otherwise *some default*.” There is never any reason to specialize `std::allocator_traits` because any trait can be overridden simply by defining the appropriate member within the specific allocator class template. Unfortunately, the standard allows such user specialization and even implies that it is a reasonable thing to do. This allowance prevents the Standards Committee from adding new members to `std::allocator_traits` without breaking existing user specializations.

In [P0401R1](#), `allocate_at_least` was proposed as a static member of `std::allocator_traits` but it was changed to a free function in [P0401R2](#) following a poll in LEWG in Cologne after it was pointed out that, because `std::allocator_traits` can be specialized and that existing specializations would not have the `allocate_at_least` member. It is this free function that is now in the September 2022 C++23 CD, [N4919](#). The current state of affairs, then, is that accessing an allocator is starting to become a hodgepodge of `std::allocator_traits` member-function calls and free-function calls. Before we standardize C++23, we should make an attempt to prevent this divergence.

Proposed resolution

This proposed resolution would disallow user specializations of `std::allocator_traits`. This change would be a breaking one, as existing specializations would become non-conforming. However, with the exception of the new `allocate_at_least` feature, existing code should continue to work for the time being. It is expected that specializations of `std::allocator_traits` are very rare, so the amount of potential breakage should be quite limited. Indeed, C++17 already added a member, `is_always_equal` to `std::allocator_traits`, theoretically breaking existing user-defined specializations but producing no outcry from the C++ community.

Wording

Changes are relative to the 2022-09-05 Working Draft, [N4917](#):

Modify section 16.4.4.6.1 [allocator.requirements.general], paragraph 3 as follows:

The class template `allocator_traits` (20.2.9) supplies a uniform interface to all allocator types. This subclause describes the requirements on allocator types and thus on types used to instantiate `allocator_traits`. A requirement is optional if a default for a given type or expression is specified. Within the standard library `allocator_traits` template, an optional requirement that is not supplied by an allocator is replaced by the specified default type or expression. ~~A user specialization of `allocator_traits` may provide different defaults and may provide defaults for different requirements than the primary template.~~ [*Note: There are no program-defined specializations of `allocator_traits`.* – end note]

And Paragraphs 43 to 46 as follows:

`a.allocate_at_least(n)`

Result: `allocation_result<X::pointer, X::size_type>`

Returns: `allocation_result<X::pointer, X::size_type>`{`ptr`, `count`} where `ptr` is memory allocated for an array of `count` `T` and such an object is created but array elements are not constructed, such that `count >= n`. If `n == 0`, the return value is unspecified.

Throws: `allocate_at_least` may throw an appropriate exception.

Remarks: ~~An allocator need not support `allocate_at_least`, but no default is provided in `allocator_traits`. If an allocator has an `allocate_at_least` member, it shall satisfy the requirements.~~
Default: `{a.allocate(n), n}`.

In section 20.2.2 [memory.syn], add a `SizeType` parameter to `allocation_result` and remove the non-member declaration for `allocate_at_least`:

```
template<class Pointer, class SizeType = size_t>
struct allocation_result {
    Pointer ptr;
    size_t SizeType count;
};
```

```
template<class Allocator>
[[nodiscard]] constexpr allocation_result<typename allocator_traits<Allocator>::pointer>
allocate_at_least(Allocator& a, size_t n); // freestanding
```

In section 20.2.9.1 [allocator.traits.general], amend the introductory paragraph:

The class template `allocator_traits` supplies a uniform interface to all allocator types. An allocator cannot be a non-class type, however, even if `allocator_traits` supplies the entire required interface. **If a program declares an explicit or partial specialization of `allocator_traits`, the program is ill-formed, no diagnostic required.**

Within the class definition, add a new member to `allocator_traits`, probably immediately before `deallocate`:

```
template<class Allocator>
[[nodiscard]] static constexpr allocation_result<pointer, size_type>
allocate_at_least(Allocator& a, size_type n);
```

And in section 20.2.9.3 [allocator.traits.members], add its definition

```
template<class Allocator>
[[nodiscard]] static constexpr allocation_result<pointer, size_type>
allocate_at_least(Allocator& a, size_type n);
```

Returns: `a.allocate_at_least(n)` if that expression is well-formed; otherwise, `{a.allocate(n), n}`.

In section 20.2.9.4 [allocator.traits.other] paragraph 2, remove the definition of non-member `allocate_at_least`:

```
template<class Allocator>
[[nodiscard]] constexpr allocation_result<typename allocator_traits<Allocator>::pointer>
allocate_at_least(Allocator& a, size_t n);
```

~~**Returns:** `a.allocate_at_least(n)` if that expression is well-formed; otherwise, `{a.allocate(n), n}`.~~

Finally, add a new subclause “Clause 20: memory management library [diff.cpp20.alloc.traits]” between [diff.cpp20.concepts] and [diff.cpp20.utilities] in Annex C:

Affected subclause: [allocator.traits.general]

Change: Forbid partial and explicit program-defined specializations of `allocator_traits`.

Rationale: Allow addition of `allocate_at_least` to `allocator_traits`, and potentially other members in the future.

Effect on original feature: Valid C++ 2020 code that partially or explicitly specializes `allocator_traits` is ill-formed with no diagnostic required in this revision of C++.

Conclusion

The current working draft has allocator operations specified as a mixture of `allocator_traits` members and namespace-scope traits. If we want to prevent this divergence, then user specializations must be disallowed in C++23, *before* the standard starts drifting in the that direction.

References

[N4917](#): Working Draft, Programming Languages – C++, 2022-09-05.

[N4919](#): Programming Languages – C++, Committee Draft, 2022-09-05.

[P0401R6](#): Providing size feedback in the Allocator interface, Jonathan Wakely and Chris Kennelly, 2021-01-22